

BÀI MỞ ĐẦU: TỔNG QUAN VỀ WEB VÀ MÔI TRƯỜNG THIẾT KẾ

1. Mục tiêu của bài:

- Nhận biết được các khái niệm cơ bản về web, website, trang web, trình duyệt, máy chủ web.
- Hiểu được mô hình client–server và vai trò của HTML, CSS, JavaScript trong thiết kế web.
- Thiết lập được môi trường làm việc cho thiết kế web (trình soạn thảo mã, trình duyệt, các công cụ hỗ trợ).

2. Nội dung bài:

2.1. Tổng quan về web và website

2.1.1. Khái niệm Internet, World Wide Web, website, trang web

Khi bắt đầu học thiết kế web, chúng ta thường có thói quen dùng các thuật ngữ *Internet*, *Web*, *website* và *trang web* như những từ đồng nghĩa. Đây là một nhầm lẫn rất phổ biến, bởi các khái niệm này có mối liên hệ rất gần gũi với nhau trong thực tế sử dụng hằng ngày. Tuy nhiên, nếu xét dưới góc độ chuyên môn, mỗi thuật ngữ lại chỉ một thành phần khác nhau trong hệ thống công nghệ thông tin. Việc phân biệt rõ các khái niệm này là nền tảng quan trọng để học tốt các nội dung tiếp theo như cấu trúc website, giao thức truyền dữ liệu, thiết kế giao diện và phát triển ứng dụng web.

Trước hết, **Internet** được hiểu là hệ thống mạng máy tính toàn cầu, trong đó vô số máy tính, thiết bị di động, máy chủ và các hệ thống mạng cục bộ được liên kết với nhau thông qua các chuẩn giao tiếp chung. Nói một cách dễ hiểu, Internet chính là cơ sở hạ tầng kết nối, cho phép các thiết bị ở nhiều nơi trên thế giới có thể trao đổi dữ liệu với nhau. Nhờ có Internet, chúng ta mới có thể gửi thư điện tử, gọi video, truyền tệp, học trực tuyến, sử dụng mạng xã hội và truy cập các dịch vụ web.

Trên nền hạ tầng Internet đó, **World Wide Web**, thường được gọi ngắn gọn là **Web**, là một dịch vụ rất phổ biến cho phép người dùng truy cập các tài liệu siêu văn bản và nhiều loại tài nguyên số khác thông qua trình duyệt. Web hoạt động dựa trên các địa chỉ định danh tài nguyên, thường là URL, cùng với các giao thức truyền dữ liệu như HTTP hoặc HTTPS. Có thể nói rằng Web không phải là toàn bộ Internet, mà chỉ là một trong nhiều dịch vụ vận hành trên Internet. Ngoài Web, Internet còn bao gồm nhiều dịch vụ khác như thư điện tử, truyền tệp, hội thoại trực tuyến hay điện toán đám mây.

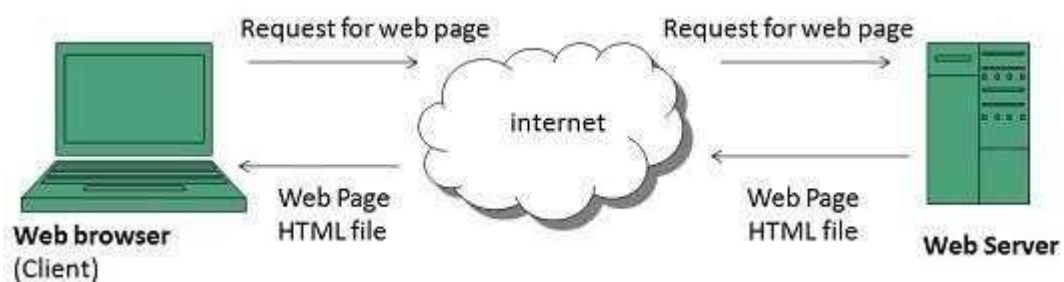
Tiếp theo, **website** là một tập hợp có tổ chức của nhiều trang thông tin và các tài nguyên liên quan, được đặt dưới cùng một tên miền hoặc do cùng một đơn

vị quản lý. Một website thường bao gồm nhiều thành phần như trang chủ, trang giới thiệu, trang liên hệ, trang sản phẩm, hình ảnh, tệp định kiểu, mã lệnh và cơ sở dữ liệu. Vì vậy, khi nói đến một website, chúng ta không chỉ nói đến một trang duy nhất, mà đang nói đến cả một hệ thống nội dung và chức năng được xây dựng để phục vụ một mục đích nhất định.

Trong khi đó, **trang web** hay **web page** là một đơn vị nội dung cụ thể nằm bên trong website. Mỗi trang web thường là một tài liệu đơn lẻ, chủ yếu được tạo bằng ngôn ngữ HTML và được trình duyệt hiển thị cho người dùng. Chẳng hạn, trong một website của trường học, trang chủ là một trang web, trang thông báo là một trang web khác, và trang thông tin tuyển sinh cũng là một trang web riêng. Như vậy, có thể hiểu website là một tập hợp, còn trang web là từng phần tử cụ thể bên trong tập hợp đó.

Để người dùng có thể truy cập và xem được nội dung web, cần có **trình duyệt web (web browser)**. Trình duyệt là phần mềm trung gian giúp gửi yêu cầu từ máy người dùng đến máy chủ web, nhận dữ liệu phản hồi và chuyển dữ liệu đó thành giao diện trực quan để người dùng quan sát và tương tác. Các trình duyệt phổ biến hiện nay như Google Chrome, Mozilla Firefox hoặc Microsoft Edge không chỉ hiển thị văn bản mà còn xử lý hình ảnh, video, âm thanh, biểu mẫu và mã lệnh phía trình duyệt như JavaScript. Nhờ trình duyệt, nội dung web trở nên sinh động, trực quan và dễ sử dụng hơn.

Bên phía cung cấp dữ liệu, **máy chủ web (web server)** là hệ thống máy tính hoặc phần mềm chuyên dùng để lưu trữ, quản lý và phân phối các tài nguyên web cho người dùng khi có yêu cầu truy cập. Khi người dùng nhập một địa chỉ web vào trình duyệt, trình duyệt sẽ gửi yêu cầu đến máy chủ tương ứng. Máy chủ sau đó trả về nội dung như tệp HTML, CSS, JavaScript, hình ảnh hoặc dữ liệu khác để trình duyệt hiển thị. Điều này cho thấy hoạt động của web luôn là sự phối hợp giữa phía người dùng và phía máy chủ.



Hình 1.1: Nguyên tắc hoạt động WWW

Bảng sau sẽ tóm tắt các khái niệm cơ bản vừa đề cập

Khái niệm	Giải thích
Internet	Hạ tầng kết nối mạng toàn cầu.
World Wide Web (Web)	Một dịch vụ phổ biến trên Internet, dựa vào URL, HTTP và trình duyệt.
Website	Tập hợp nhiều trang web và tài nguyên liên quan.
Trang web (web page)	Một trang tài liệu cụ thể, thường là tệp HTML được trình duyệt hiển thị.
Trình duyệt	Phần mềm dùng để truy cập, tải và hiển thị nội dung web.
Máy chủ web	Máy tính/phần mềm lưu trữ và phân phối tài nguyên web cho client.

Bảng 1.1: Các khái niệm Internet, World Wide Web, website, trang web

Bài thực hành sau giúp phân biệt website và trang web (web page)

Thực hành 1: Phân biệt website và trang web

Bước 1. Mở trình duyệt và truy cập trang chủ của một trường học hoặc báo điện tử.

Bước 2. Quan sát thanh địa chỉ: tên miền đại diện cho website; nội dung đang mở trên màn hình là một trang web cụ thể.

Bước 3. Nhấn vào 2–3 liên kết khác nhau trong cùng site.

Bước 4. So sánh URL trước và sau khi nhấp liên kết để nhận ra website giữ nguyên tên miền, còn trang web thay đổi theo đường dẫn.

Bước 5. Ghi lại kết luận bằng 2 câu: website là gì, trang web là gì.

2.1.2. Tên miền (domain), hosting, máy chủ web, địa chỉ URL

Trong quá trình học thiết kế và phát triển web, chúng ta sẽ thường xuyên gặp các thuật ngữ như **tên miền**, **hosting**, **máy chủ web** và **URL**. Đây là những khái niệm nền tảng, xuất hiện ngay từ giai đoạn xây dựng website đầu tiên cho đến khi triển khai website ra môi trường thực tế. Nếu không hiểu rõ bản chất của từng thành phần, chúng ta rất dễ nhầm lẫn giữa “địa chỉ truy cập”, “nơi lưu trữ”, “hệ thống phục vụ dữ liệu” và “đường dẫn cụ thể đến tài nguyên”. Vì vậy, việc nắm chắc các khái niệm này không chỉ giúp học tốt lý thuyết mà còn hỗ trợ rất nhiều cho thực hành triển khai website sau này.

Trước hết, **tên miền (domain name)** là địa chỉ có ý nghĩa và dễ ghi nhớ để con người sử dụng khi truy cập một website hoặc một dịch vụ trên web. Thay vì phải nhớ một dãy số địa chỉ IP phức tạp của máy chủ, người dùng chỉ cần nhập một tên quen thuộc như *mozilla.org* hoặc *developer.mozilla.org* vào trình duyệt. Có thể hiểu tên miền như “địa chỉ chữ” đại diện cho một hệ thống trên Internet.

Nhờ cơ chế phân giải tên miền, trình duyệt có thể chuyển tên miền mà con người nhập vào thành địa chỉ IP tương ứng của máy chủ để kết nối đến đúng nơi cần truy cập. Vì vậy, tên miền giữ vai trò rất quan trọng trong việc giúp website trở nên dễ nhớ, thuận tiện khi sử dụng và chuyên nghiệp hơn trong môi trường số.

Về mặt cấu trúc, tên miền thường được tổ chức theo từng cấp. Chẳng hạn, trong địa chỉ *developer.mozilla.org*, phần *org* là miền cấp cao nhất, *mozilla* là tên miền chính, còn *developer* là miền con. Cách tổ chức này cho phép một tổ chức có thể chia nội dung hoặc dịch vụ thành nhiều khu vực khác nhau mà vẫn nằm trong cùng hệ thống tên miền chung.

Tiếp theo, **hosting** là dịch vụ cung cấp không gian lưu trữ và hạ tầng cần thiết để website có thể hoạt động trên Internet. Khi xây dựng một website, chúng ta tạo ra nhiều tệp và tài nguyên như tệp HTML, CSS, JavaScript, hình ảnh, video hoặc cơ sở dữ liệu. Những thành phần này cần được đặt ở một nơi có kết nối mạng ổn định để người dùng ở nhiều nơi khác nhau có thể truy cập được. Hosting chính là dịch vụ đáp ứng nhu cầu đó. Nói một cách dễ hiểu, nếu website được xem như một ngôi nhà số, thì hosting chính là mảnh đất hoặc nơi đặt ngôi nhà ấy để mọi người có thể đến thăm.

Trong thực tế, hosting có nhiều hình thức khác nhau. Có loại **shared hosting**, trong đó nhiều website cùng sử dụng chung tài nguyên của một máy chủ; có **VPS** là máy chủ ảo riêng với mức độ chủ động và hiệu năng cao hơn; có **cloud server** với khả năng mở rộng linh hoạt; và cũng có những nền tảng triển khai chuyên biệt dành cho từng loại ứng dụng hoặc công nghệ nhất định. Mỗi mô hình hosting có ưu điểm, hạn chế và chi phí khác nhau. Tuy nhiên, dù ở hình thức nào, bản chất của hosting vẫn là cung cấp môi trường để lưu trữ website và cho phép website sẵn sàng phục vụ người truy cập qua mạng.

Liên quan chặt chẽ với hosting là khái niệm **máy chủ web**. Máy chủ web có thể được hiểu theo hai góc độ. Ở góc độ phần cứng, đó là một máy tính hoặc một hệ thống máy tính có nhiệm vụ lưu trữ và cung cấp tài nguyên web. Ở góc độ phần mềm, đó là chương trình chuyên nhận các yêu cầu từ trình duyệt hoặc từ ứng dụng khách, sau đó phản hồi bằng nội dung phù hợp thông qua các giao thức như HTTP hoặc HTTPS. Khi người dùng nhập địa chỉ website vào trình duyệt, trình duyệt sẽ gửi yêu cầu đến máy chủ web. Máy chủ web sẽ xử lý yêu cầu này và trả về các tài nguyên tương ứng như trang HTML, tệp định kiểu CSS, mã lệnh JavaScript, hình ảnh, video hoặc dữ liệu ở dạng JSON.

Một khái niệm rất quan trọng khác là **URL**. URL là viết tắt của *Uniform Resource Locator*, có thể hiểu là địa chỉ đầy đủ của một tài nguyên trên Internet.

Nếu tên miền chỉ cho biết địa chỉ chung của một hệ thống, thì URL giúp xác định chính xác người dùng đang muốn truy cập tài nguyên nào trong hệ thống đó. Một URL thường bao gồm nhiều thành phần như giao thức, tên miền, đường dẫn đến tài nguyên và trong một số trường hợp có thêm chuỗi truy vấn hoặc phần neo để xác định vị trí cụ thể trong tài liệu.

Ví dụ, trong một địa chỉ đầy đủ, phần đầu như *http* hoặc *https* cho biết giao thức được sử dụng để truyền dữ liệu. Phần tiếp theo là tên miền, cho biết máy chủ hoặc hệ thống đích cần truy cập. Sau đó là **đường dẫn**, dùng để chỉ ra thư mục, trang hoặc tài nguyên cụ thể trên máy chủ. Ngoài ra, URL còn có thể chứa **chuỗi truy vấn**, thường dùng để truyền tham số, chẳng hạn như từ khóa tìm kiếm, mã định danh hoặc các điều kiện lọc dữ liệu. Cuối cùng, một số URL có thêm **phần neo**, giúp đưa người dùng đến một vị trí nhất định bên trong trang web thay vì chỉ mở trang từ đầu.

Bảng sau sẽ tóm tắt các khái niệm cơ bản vừa đề cập

Khái niệm	Giải thích
Tên miền (domain name)	Địa chỉ có ý nghĩa và dễ ghi nhớ để con người sử dụng khi truy cập một website hoặc một dịch vụ trên web
Hosting	Dịch vụ cung cấp không gian lưu trữ và hạ tầng cần thiết để website có thể hoạt động trên Internet
URL (Uniform Resource Locator)	Địa chỉ đầy đủ của một tài nguyên trên Internet

Bảng 1.2: Tóm tắt các khái niệm tên miền, hosting, máy chủ web, URL

Ví dụ 1 phân tích cấu trúc URL: <https://developer.mozilla.org/en-US/docs/Web/HTML>

- Giao thức (protocol): https
- Tên miền: developer.mozilla.org
- Đường dẫn đến tài nguyên: /en-US/docs/Web/HTML

Ví dụ 2 phân tích cấu trúc URL:

<https://example.com/products/list?page=2#top>

- Giao thức: https
- Tên miền: example.com
- Đường dẫn: /products/list
- Tham số truy vấn: page=2
- Fragment: top

Thực hành 2: Đọc các thành phần của một URL

Bước 1. Sao chép URL của một trang tài liệu MDN đang mở.

Bước 2. Gạch chân giao thức ở đầu URL.

Bước 3. Khoanh tên miền chính.

Bước 4. Tách phần đường dẫn phía sau tên miền.

Bước 5. Nếu có dấu ?, hãy xác định phần query string; nếu có dấu #, hãy xác định phần fragment.

Bước 6. Trao đổi trong nhóm: phần nào của URL thay đổi khi sang bài viết khác, phần nào thường giữ nguyên?

2.1.3. Giao thức HTTP/HTTPS và vai trò của trình duyệt web

a. Giao thức HTTP/HTTPS

Khi nghiên cứu về web, một trong những nội dung nền tảng mà chúng ta cần nắm vững là cách trình duyệt và máy chủ giao tiếp với nhau để đưa một trang web từ môi trường mạng lên màn hình của người dùng. Đằng sau thao tác rất quen thuộc như nhập địa chỉ website và nhấn Enter là cả một quá trình trao đổi dữ liệu có tổ chức, trong đó **HTTP**, **HTTPS** và **trình duyệt web** giữ vai trò hết sức quan trọng. Hiểu được cơ chế này sẽ giúp chúng ta có cái nhìn đúng bản chất hoạt động của web, thay vì chỉ tiếp cận web như một giao diện trực quan đơn thuần.

Trước hết, **HTTP** là viết tắt của *HyperText Transfer Protocol*, tức giao thức truyền tải siêu văn bản. Đây là giao thức nền tảng được sử dụng để trao đổi tài nguyên trên World Wide Web. Nói một cách dễ hiểu, HTTP chính là bộ quy tắc giúp trình duyệt và máy chủ “nói chuyện” được với nhau. Khi người dùng yêu cầu truy cập một trang web, trình duyệt sẽ gửi một thông điệp yêu cầu đến máy chủ. Máy chủ tiếp nhận yêu cầu này, xử lý nội dung cần thiết, sau đó gửi lại một thông điệp phản hồi chứa tài nguyên hoặc dữ liệu tương ứng. Chính nhờ cơ chế đó mà thông tin trên web có thể được truyền từ nơi lưu trữ đến thiết bị của người dùng.

Trong mô hình này, trình duyệt thường được gọi là **client**, còn máy chủ là **server**. Client là phía khởi tạo yêu cầu, còn server là phía tiếp nhận và cung cấp phản hồi. Đây là mô hình **yêu cầu – đáp ứng** (*request – response*), một mô hình đặc biệt quan trọng trong phát triển web. Mỗi lần người dùng mở một trang, nhấn vào một liên kết, gửi biểu mẫu, đăng nhập tài khoản hay thực hiện thao tác tìm kiếm, trình duyệt đều phải tạo ra yêu cầu gửi đến máy chủ. Máy chủ dựa vào nội dung yêu cầu đó để quyết định sẽ trả về trang HTML, dữ liệu JSON, hình ảnh, tệp

CSS hay một thông báo lỗi. Vì vậy, có thể nói mô hình yêu cầu – đáp ứng là nguyên lý vận hành cốt lõi của hầu hết các ứng dụng web hiện nay.

Từ nền tảng HTTP, **HTTPS** ra đời như một phiên bản an toàn hơn cho việc truyền dữ liệu trên web. HTTPS thực chất là HTTP được kết hợp với một lớp bảo mật mã hóa trong quá trình trao đổi thông tin giữa trình duyệt và máy chủ. Ở mức nhập môn, chúng ta chưa cần đi sâu vào các chi tiết kỹ thuật phức tạp của cơ chế mã hóa, nhưng cần ghi nhớ một điểm cốt lõi: **HTTPS giúp bảo vệ dữ liệu khi truyền trên mạng và làm tăng độ tin cậy của website**. Nhờ có HTTPS, những thông tin quan trọng như mật khẩu, dữ liệu cá nhân, nội dung biểu mẫu hoặc giao dịch trực tuyến được giảm nguy cơ bị nghe lén, giả mạo hoặc can thiệp trên đường truyền. Đây là lý do vì sao phần lớn các website hiện đại đều ưu tiên sử dụng HTTPS thay cho HTTP thông thường.

b. Vai trò của trình duyệt web

Trong thực tế sử dụng, nhiều người thường nghĩ rằng khi truy cập một trang web, trình duyệt chỉ tải về một tệp HTML duy nhất. Tuy nhiên, đó chỉ là một phần rất nhỏ của toàn bộ quá trình. Một trang web hiện đại thường được cấu thành từ nhiều loại tài nguyên khác nhau. Sau khi nhận được tệp HTML ban đầu, trình duyệt tiếp tục phân tích nội dung bên trong để xác định xem trang đó còn liên kết đến những tài nguyên nào khác. Các tài nguyên này có thể bao gồm **tệp CSS** dùng để định dạng giao diện, **tệp JavaScript** dùng để tạo hành vi và tương tác, **hình ảnh, font chữ, biểu tượng, video** hoặc nhiều tài nguyên bổ sung khác. Như vậy, quá trình tải một trang web hoàn chỉnh thực chất là quá trình tải và phối hợp nhiều thành phần chứ không chỉ đơn giản là nhận một văn bản HTML.

Điều này giải thích vì sao một website có thể hiển thị chậm nếu chứa quá nhiều hình ảnh nặng, nhiều tệp mã lệnh hoặc nhiều tài nguyên từ các nguồn khác nhau. Trình duyệt không chỉ nhận dữ liệu một lần rồi kết thúc, mà phải liên tục gửi thêm các yêu cầu mới để lấy từng tài nguyên cần thiết. Mỗi tài nguyên được tải về sẽ góp phần hoàn thiện giao diện và chức năng của trang. Từ góc nhìn sơ phạm, đây là điểm rất quan trọng để chúng ta hiểu rằng thiết kế web không phải chỉ là viết HTML, mà là tổ chức nhiều loại thành phần sao cho chúng phối hợp hiệu quả với nhau.

Không dừng lại ở việc tải dữ liệu, **trình duyệt** còn thực hiện hàng loạt công việc xử lý bên trong để biến các tệp nhận được thành một trang web mà người dùng có thể quan sát và tương tác. Trước hết, trình duyệt đọc tài liệu HTML và **phân tích cú pháp** để tạo thành một cấu trúc dạng cây gọi là **DOM (Document Object Model)**. Có thể hiểu DOM là mô hình biểu diễn toàn bộ các phần tử trong

trang web dưới dạng các nút có quan hệ cha, con và anh em. Nhờ có cấu trúc này, trình duyệt biết được tiêu đề nằm ở đâu, đoạn văn nào xuất hiện trước, hình ảnh nào thuộc khu vực nào, và phần tử nào đang được lồng bên trong phần tử nào.

Sau khi đã có cấu trúc nội dung từ HTML, trình duyệt tiếp tục đọc các quy tắc trong CSS để xác định cách hiển thị từng phần tử. CSS giúp quyết định màu sắc, kích thước chữ, khoảng cách, đường viền, vị trí bố cục, cách căn lề và nhiều yếu tố trình bày khác. Nói cách khác, nếu HTML tạo nên “bộ khung nội dung”, thì CSS làm cho bộ khung đó trở nên có hình thức rõ ràng, thẩm mỹ và đúng bố cục mong muốn. Khi trình duyệt áp dụng CSS lên DOM, giao diện của trang bắt đầu có dáng vẻ mà người thiết kế hướng đến.

Song song với đó, nếu trang web có sử dụng **JavaScript**, trình duyệt sẽ thực thi các đoạn mã này để tạo ra hành vi tương tác. Nhờ JavaScript, trang web có thể phản hồi khi người dùng nhấn nút, nhập dữ liệu, kéo thả, mở hộp thoại, gửi biểu mẫu mà không cần tải lại toàn bộ trang, hoặc cập nhật nội dung theo thời gian thực. Chính JavaScript làm cho web hiện đại trở nên linh hoạt và sinh động hơn rất nhiều so với những trang chỉ hiển thị nội dung tĩnh.

Từ đó có thể thấy rằng trình duyệt không chỉ là công cụ “mở web”, mà là một môi trường phần mềm phức tạp có khả năng tiếp nhận dữ liệu, phân tích cấu trúc, áp dụng định dạng, thực thi mã lệnh và hiển thị kết quả cuối cùng cho người dùng. Toàn bộ quá trình này diễn ra rất nhanh, thường chỉ trong vài giây hoặc thậm chí trong tích tắc, khiến người dùng có cảm giác như trang web xuất hiện gần như ngay lập tức.

Để hệ thống hóa nội dung, có thể tóm tắt như sau:

Khái niệm	Nội dung cốt lõi
HTTP	Giao thức nền tảng dùng để trao đổi tài nguyên giữa trình duyệt và máy chủ trên Web.
HTTPS	Phiên bản bảo mật của HTTP, có thêm cơ chế mã hóa để bảo vệ dữ liệu truyền đi.
Mô hình yêu cầu – đáp ứng	Trình duyệt gửi yêu cầu, máy chủ xử lý và trả về phản hồi phù hợp.
Tài nguyên web	Không chỉ có HTML mà còn gồm CSS, JavaScript, hình ảnh, font chữ, video và dữ liệu khác.
DOM	Cấu trúc cây biểu diễn nội dung HTML để trình duyệt quản lý và hiển thị trang.

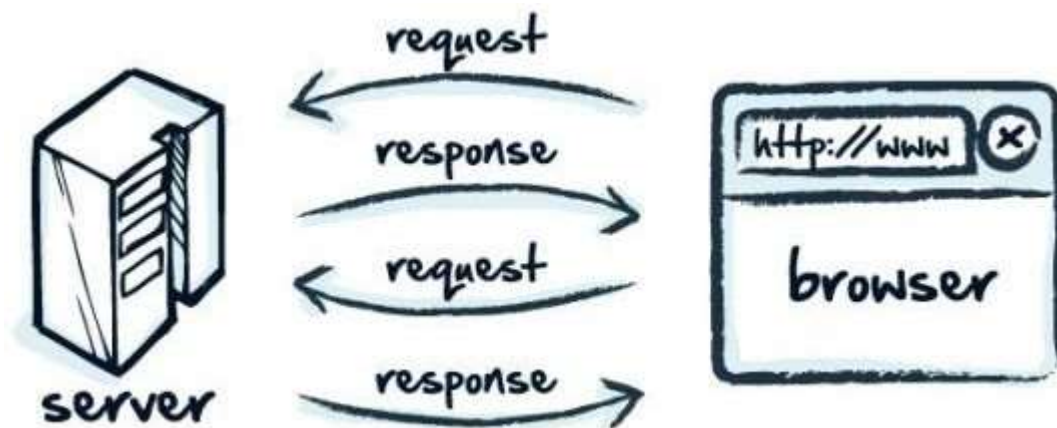
Vai trò của CSS	Tạo bố cục, màu sắc, kiểu dáng và hình thức trình bày cho nội dung.
Vai trò của JavaScript	Tạo hành vi, xử lý tương tác và làm cho trang web hoạt động linh hoạt hơn.

Bảng 1.3: Tóm tắt các khái niệm giao thức HTTP/HTTPS và vai trò của trình duyệt web

2.2. Mô hình client–server trong thiết kế web

2.2.1. Mối quan hệ giữa client (trình duyệt) và server (máy chủ)

Trong thiết kế web, để một trang thông tin hay một ứng dụng web có thể hoạt động, luôn tồn tại sự phối hợp giữa hai phía cơ bản là **client** và **server**. Đây là mô hình tổ chức nền tảng của hầu hết các hệ thống web hiện đại. Việc hiểu đúng mối quan hệ giữa hai thành phần này giúp chúng ta không chỉ biết “trang web hiển thị như thế nào”, mà còn hiểu được “vì sao nó hiển thị được”, “dữ liệu đến từ đâu” và “ai chịu trách nhiệm xử lý từng phần công việc trong hệ thống”.



Hình 1.2: Mối quan hệ giữa client (browser) và server

Client là phía trực tiếp tương tác với người dùng. Trong bối cảnh web, client thường chính là **trình duyệt** như Google Chrome, Mozilla Firefox, Microsoft Edge hoặc Safari. Trình duyệt tiếp nhận thao tác của người dùng, chẳng hạn nhập địa chỉ web, nhấn vào liên kết, điền biểu mẫu, bấm nút đăng nhập hay thực hiện tìm kiếm. Sau khi nhận thao tác, trình duyệt chuyển các thao tác đó thành yêu cầu kỹ thuật để gửi đến hệ thống ở phía bên kia mạng. Vì vậy, client là thành phần đại diện cho phía sử dụng dịch vụ web.

Ngược lại, **server** là phía cung cấp dịch vụ. Về bản chất, server là hệ thống máy tính hoặc phần mềm có nhiệm vụ tiếp nhận yêu cầu từ client, xử lý yêu cầu đó và gửi lại kết quả phù hợp. Máy chủ thường là nơi lưu trữ website, quản lý tài

nguyên, vận hành mã xử lý phía sau và kết nối với cơ sở dữ liệu khi cần. Nếu client là nơi phát sinh nhu cầu truy cập, thì server là nơi đáp ứng nhu cầu ấy bằng tài nguyên hoặc dữ liệu cụ thể.

Mối quan hệ giữa client và server được hình thành trên cơ sở **phân công chức năng rõ ràng**. Trình duyệt không lưu trữ toàn bộ nội dung hệ thống cũng không tự quyết định dữ liệu chính thức của website. Nó chủ yếu đảm nhận việc gửi yêu cầu, nhận phản hồi và hiển thị kết quả cho người dùng. Máy chủ không trực tiếp thay người dùng thao tác trên giao diện, nhưng lại giữ vai trò trung tâm trong việc cung cấp nội dung, xử lý nghiệp vụ và kiểm soát dữ liệu. Nhờ sự phân chia này, hệ thống web có thể vận hành theo hướng chuyên biệt: phía client tập trung vào trải nghiệm sử dụng, còn phía server chịu trách nhiệm về logic và dữ liệu.

Cần nhấn mạnh rằng client và server không tồn tại như hai thành phần tách rời, mà luôn **phụ thuộc lẫn nhau trong quá trình cung cấp dịch vụ web**. Nếu không có client gửi yêu cầu, server không biết người dùng đang cần gì. Nếu không có server xử lý và phản hồi, client không thể có đủ dữ liệu để dựng nên trang web hoàn chỉnh. Điều này cho thấy mối quan hệ giữa hai phía là mối quan hệ phối hợp có trình tự, trong đó mỗi thành phần giữ một vai trò riêng nhưng cùng hướng tới mục tiêu chung là cung cấp nội dung và chức năng cho người sử dụng.

Từ góc độ kỹ thuật, trình duyệt và máy chủ giao tiếp với nhau thông qua các **giao thức mạng**, phổ biến nhất là HTTP và HTTPS. Giao thức đóng vai trò như bộ quy tắc chung giúp hai phía hiểu được cách gửi, nhận và diễn giải dữ liệu. Nhờ có các quy tắc chuẩn này, một trình duyệt trên máy tính cá nhân, điện thoại thông minh hay máy tính bảng vẫn có thể giao tiếp được với máy chủ web đặt ở bất kỳ đâu trên Internet. Đây là yếu tố quan trọng bảo đảm tính tương thích và khả năng mở rộng của các hệ thống web.

Trong thiết kế web hiện đại, phía client và phía server còn khác nhau về phạm vi xử lý. Ở phía client, trình duyệt thường đảm nhiệm việc hiển thị bố cục, áp dụng kiểu trình bày, tiếp nhận thao tác chuột và bàn phím, đồng thời thực thi các đoạn mã JavaScript chạy trên máy người dùng để tăng tính tương tác. Ở phía server, hệ thống xử lý các công việc mang tính trung tâm hơn như kiểm tra đăng nhập, xác thực người dùng, phân quyền truy cập, truy vấn cơ sở dữ liệu, lưu thông tin và thực hiện các quy tắc nghiệp vụ. Cách tổ chức này giúp website vừa phản hồi linh hoạt ở giao diện, vừa bảo đảm độ tin cậy trong xử lý dữ liệu.

Chúng ta cũng cần phân biệt rõ rằng **client không đồng nhất với người dùng**, còn **server không chỉ đơn giản là một máy tính đặt từ xa**. Người dùng

là chủ thể thao tác, nhưng client là phần mềm đại diện cho phía người dùng trong giao tiếp kỹ thuật. Tương tự, server có thể là một máy tính vật lý, một máy chủ ảo, hoặc một hệ thống nhiều máy chủ phối hợp với nhau; điều quan trọng không phải là hình thức phần cứng, mà là vai trò cung cấp dịch vụ và xử lý yêu cầu trong mô hình web.

Bảng sau tóm tắt các khái niệm

Khái niệm	Giải thích
Client	Phía yêu cầu dịch vụ trong hệ thống web; trong bối cảnh học nhập môn thường chính là trình duyệt trên thiết bị của người dùng.
Trình duyệt web	Phần mềm tiếp nhận thao tác người dùng, gửi yêu cầu đến máy chủ, nhận phản hồi và dựng nội dung để hiển thị trên màn hình.
Server	Phía cung cấp dịch vụ; tiếp nhận yêu cầu từ client, xử lý dữ liệu và trả về tài nguyên hoặc kết quả phù hợp.
Máy chủ web	Hệ thống phần cứng hoặc phần mềm vận hành website, lưu trữ và phân phối các tài nguyên web cho client.
Yêu cầu (Request)	Thông điệp do client gửi đến server nhằm xin một tài nguyên hoặc yêu cầu thực hiện một thao tác xử lý.
Phản hồi (Response)	Thông điệp do server gửi lại cho client, chứa tài nguyên, dữ liệu hoặc trạng thái xử lý.
HTTP/HTTPS	Giao thức giao tiếp giữa client và server trên web; HTTPS bổ sung lớp bảo mật mã hóa trong quá trình truyền dữ liệu.
Tài nguyên web	Các thành phần mà server cung cấp cho client như HTML, CSS, JavaScript, hình ảnh, video hoặc dữ liệu JSON.
Mô hình client-server	Mô hình tổ chức trong đó client khởi tạo yêu cầu, còn server tiếp nhận, xử lý và phản hồi kết quả.

Bảng 1.4: Tóm tắt các khái niệm mối quan hệ giữa client (trình duyệt) và server (máy chủ)

Bài thực hành: Quan sát mối quan hệ giữa client và server bằng trình duyệt

Bước 1. Mở trình duyệt web

Khởi động Google Chrome hoặc Microsoft Edge trên máy tính.

Bước 2. Truy cập một website

Nhập địa chỉ một website đơn giản, ví dụ một trang trường học, trang tin tức hoặc một website giới thiệu doanh nghiệp, rồi nhấn Enter.

Bước 3. Quan sát kết quả hiển thị

Nhận xét rằng sau khi nhập địa chỉ, trang web xuất hiện trên màn hình. Tự trả lời các câu hỏi: dữ liệu này đến từ đâu, và trình duyệt đã làm gì để lấy được dữ liệu đó.

Bước 4. Mở công cụ dành cho nhà phát triển

Nhấn phím F12 hoặc nhấp chuột phải trên trang và chọn **Inspect** để mở DevTools.

Bước 5. Chọn thẻ Network

Trong DevTools, chọn tab **Network**. Đây là nơi hiển thị các yêu cầu mà trình duyệt gửi đến máy chủ.

Bước 6. Tải lại trang web

Nhấn phím F5 hoặc nút tải lại trang. Quan sát danh sách các yêu cầu xuất hiện trong thẻ Network.

Bước 7. Xác định tài nguyên đầu tiên

Tìm yêu cầu đầu tiên, thường là tài liệu HTML chính của trang.

Bước 8. Quan sát các tài nguyên bổ sung

Tiếp tục xem các mục như CSS, JavaScript, hình ảnh, font chữ. Từ đó rút ra rằng trình duyệt không chỉ lấy một tệp HTML mà còn yêu cầu thêm nhiều tài nguyên khác để dựng trang hoàn chỉnh.

Bước 9. Chọn một yêu cầu cụ thể để xem chi tiết

Nhấp vào một yêu cầu trong danh sách. Quan sát các thông tin như địa chỉ yêu cầu, phương thức gửi, mã trạng thái và loại nội dung phản hồi.

Bước 10. Liên hệ với lý thuyết client-server

Trả lời: trình duyệt là phía nào, máy chủ là phía nào, dữ liệu nào được yêu cầu, dữ liệu nào được trả về.

Bước 11. Ghi chép kết quả

Lập một bảng ngăn gồm các cột: Tên tài nguyên, loại tài nguyên, ai gửi yêu cầu, ai trả phản hồi, vai trò của tài nguyên trong trang web.

Bước 12. Kết luận thực hành

Viết một đoạn ngắn từ 5 đến 7 câu nêu rõ: khi truy cập một website, client và server đã phối hợp với nhau như thế nào.

2.2.2. Vai trò của HTML, CSS, JavaScript phía client

Trong thiết kế web, khi nghiên cứu phía client, chúng ta cần nắm vững ba công nghệ nền tảng là **HTML**, **CSS** và **JavaScript**. Đây là ba thành phần cốt lõi tạo nên hầu hết các trang web hiện đại ở phía trình duyệt. Nếu ví website là một công trình hoàn chỉnh mà người dùng nhìn thấy và sử dụng trực tiếp trên màn hình, thì HTML tạo ra khung nội dung, CSS đảm nhận phần trình bày trực quan, còn JavaScript bổ sung khả năng xử lý và tương tác. Việc phân biệt rõ vai trò của từng công nghệ không chỉ giúp học tốt phần nhập môn mà còn tạo nền tảng vững chắc cho các nội dung tiếp theo như giao diện người dùng, thao tác với DOM, lập trình tương tác và phát triển ứng dụng web hoàn chỉnh.

a. HTML

Trước hết, **HTML** là viết tắt của *HyperText Markup Language*, tức ngôn ngữ đánh dấu siêu văn bản. HTML giữ vai trò xây dựng **cấu trúc nội dung** của trang web. Khi trình duyệt nhận được tài liệu HTML từ máy chủ, nó sẽ phân tích các thẻ đánh dấu để hiểu đâu là tiêu đề, đâu là đoạn văn, đâu là hình ảnh, đâu là liên kết, đâu là bảng biểu, đâu là biểu mẫu nhập liệu. Nói cách khác, HTML không tập trung vào việc làm cho trang web đẹp hay sinh động, mà tập trung vào việc xác định nội dung của trang là gì và các thành phần đó được tổ chức theo trật tự nào.

Chính vì giữ vai trò cấu trúc, HTML là lớp nền đầu tiên của giao diện web. Nếu một trang chỉ có HTML mà chưa có CSS và JavaScript, trình duyệt vẫn có thể hiển thị nội dung ở dạng cơ bản. Người dùng vẫn đọc được văn bản, vẫn nhấn được liên kết và vẫn điền được biểu mẫu nếu các phần tử cần thiết đã được khai báo. Điều này cho thấy HTML là điều kiện cần để một trang web tồn tại như một tài liệu có ý nghĩa. Trong giảng dạy, cần giúp chúng ta hiểu rằng HTML không phải là công cụ “trang trí”, mà là ngôn ngữ mô tả nội dung và cấu trúc logic của trang.

b. CSS

Bên trên lớp cấu trúc đó, **CSS** là viết tắt của *Cascading Style Sheets*, tức ngôn ngữ định kiểu theo tầng. CSS có nhiệm vụ quy định **cách hiển thị** của các phần tử HTML trong trình duyệt. Nếu HTML trả lời câu hỏi “trang web có những gì”, thì CSS trả lời câu hỏi “những thành phần đó được trình bày như thế nào”. Nhờ CSS, chúng ta có thể thiết lập màu sắc chữ, màu nền, kích thước phông chữ, khoảng cách giữa các phần tử, đường viền, độ rộng khối nội dung, cách căn lề, cách sắp xếp bố cục và nhiều chi tiết trực quan khác.

Vai trò của CSS không dừng ở việc làm cho giao diện đẹp hơn. Về mặt kỹ thuật, CSS giúp tách riêng **nội dung** khỏi **hình thức trình bày**. Sự tách biệt này

rất quan trọng trong thiết kế web chuyên nghiệp, bởi nó giúp mã nguồn rõ ràng hơn, dễ bảo trì hơn và thuận lợi hơn khi cần thay đổi giao diện mà không phải chỉnh sửa cấu trúc HTML quá nhiều. Một website có thể dùng cùng một nội dung HTML nhưng áp dụng nhiều bộ CSS khác nhau để tạo ra những phong cách hiển thị hoàn toàn khác biệt. Nhờ đó, website có thể đạt được cả tính thẩm mỹ, tính nhất quán và khả năng thích ứng với nhiều loại thiết bị như máy tính, máy tính bảng và điện thoại.

c. JavaScript

Nếu HTML tạo cấu trúc và CSS tạo hình thức, thì **JavaScript** giữ vai trò bổ sung **hành vi và tương tác** cho phía client. JavaScript là ngôn ngữ lập trình chạy trong trình duyệt, cho phép trang web phản hồi trước thao tác của người dùng và thay đổi nội dung trên màn hình một cách linh hoạt. Nhờ JavaScript, các nút bấm có thể xử lý sự kiện, biểu mẫu có thể kiểm tra dữ liệu trước khi gửi, menu có thể mở ra hoặc thu gọn, nội dung có thể được cập nhật mà không cần tải lại toàn bộ trang, và nhiều hiệu ứng tương tác có thể xuất hiện ngay trên giao diện.

Trong bối cảnh thiết kế web hiện đại, JavaScript giữ vai trò hết sức quan trọng vì nó biến một trang web tĩnh thành một giao diện có khả năng hoạt động. Chẳng hạn, với HTML và CSS, chúng ta có thể tạo ra một biểu mẫu đăng ký có bố cục rõ ràng và trình bày đẹp mắt. Tuy nhiên, để kiểm tra xem người dùng đã nhập đủ thông tin hay chưa, email có đúng định dạng hay không, hoặc để hiển thị thông báo lỗi ngay lập tức trên màn hình, cần có JavaScript xử lý. Từ đó, người học có thể nhận thấy rằng HTML, CSS và JavaScript không cạnh tranh vai trò với nhau mà phối hợp với nhau để tạo thành một hệ thống hoàn chỉnh ở phía client.

Mối quan hệ giữa HTML, CSS và JavaScript còn thể hiện rõ khi làm việc với **DOM**. Sau khi đọc HTML, trình duyệt xây dựng cây DOM để biểu diễn các phần tử trong trang. CSS được áp dụng lên các nút trong cây đó để xác định kiểu hiển thị. JavaScript có thể truy cập DOM, thay đổi nội dung, thêm hoặc xóa phần tử, chỉnh sửa thuộc tính và điều khiển giao diện theo thời gian thực. Đây là lý do vì sao người học cần hiểu đúng thứ tự: HTML cung cấp nền tảng để trình duyệt tạo cấu trúc, CSS điều chỉnh cách cấu trúc đó xuất hiện, còn JavaScript thao tác lên cấu trúc ấy để tạo ra sự linh hoạt trong sử dụng.

Bảng sau tóm tắt các khái niệm

Khái niệm	Vai trò kỹ thuật trong phía client
HTML	Xây dựng cấu trúc và nội dung của trang web bằng các phần tử như tiêu đề, đoạn văn, liên kết, hình ảnh, bảng và biểu mẫu.

CSS	Quy định cách hiển thị của các phần tử HTML như màu sắc, kích thước, khoảng cách, bố cục và phong cách trình bày.
JavaScript	Tạo hành vi và khả năng tương tác cho trang web, xử lý sự kiện, kiểm tra dữ liệu và thay đổi nội dung giao diện động.

Bảng 1.5: Tóm tắt các khái niệm HTML, CSS, JavaScript

Bài thực hành: Phân biệt vai trò của HTML, CSS và JavaScript trong phía client

Bước 1. Tạo thư mục làm việc

Tạo một thư mục mới với tên gợi nhớ, ví dụ bai_thuc_hanh_client.

Bước 2. Tạo tệp HTML

Trong thư mục vừa tạo, tạo tệp index.html.

Bước 3. Nhập nội dung HTML cơ bản

Mở tệp index.html và nhập nội dung sau:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Thực hành HTML CSS JavaScript</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="tieuDe">Giới thiệu sản phẩm</h1>
  <p>Sản phẩm này dùng để minh họa vai trò của HTML, CSS và JavaScript.</p>
  <button id="nutBam">Nhấn để đổi nội dung</button>
  <script src="script.js"></script>
</body>
</html>
```

Nhận xét rằng ở bước này, HTML đã tạo ra các thành phần nội dung chính của trang gồm tiêu đề, đoạn văn và nút bấm.

Bước 4. Tạo tệp CSS

Tạo tệp style.css trong cùng thư mục và nhập nội dung:

```
body {
  font-family: Arial, sans-serif;
  margin: 30px;
  background-color: #f4f7fb;
```

```
}  
h1 {  
  color: #1f4e79;  
}  
button {  
  padding: 10px 16px;  
  background-color: #2e75b6;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

Lưu ý rằng CSS không tạo thêm nội dung mới, mà chỉ làm thay đổi hình thức hiển thị của các phần tử HTML đã có.

Bước 5. Tạo tệp JavaScript

Tạo tệp script.js trong cùng thư mục và nhập nội dung:

```
document.getElementById("nutBam").addEventListener("click", function  
( ) {  
  document.getElementById("tieuDe").textContent = "Bạn vừa kích hoạt  
JavaScript";  
});
```

Chú ý rằng JavaScript đang lắng nghe sự kiện nhấn nút và thay đổi nội dung của tiêu đề khi người dùng tương tác.

Bước 6. Mở tệp HTML trong trình duyệt

Nhấp đúp vào index.html để mở bằng trình duyệt. Quan sát giao diện hiển thị.

Bước 7. Xác định vai trò của HTML

Chú ý rằng, nếu bỏ tệp CSS và JavaScript đi, trang còn lại những gì. Từ đó rút ra rằng HTML là phần tạo nên khung nội dung cơ bản.

Bước 8. Kiểm tra vai trò của CSS

Tạm thời đổi tên tệp style.css thành một tên khác, rồi tải lại trang. Quan sát sự thay đổi của giao diện. Sau đó đổi lại tên ban đầu. Chúng ta sẽ thấy rằng nội dung vẫn còn nhưng hình thức hiển thị thay đổi rõ rệt.

Bước 9. Kiểm tra vai trò của JavaScript

Tạm thời đổi tên tệp script.js thành một tên khác, rồi tải lại trang và nhấn nút. Quan sát rằng nút vẫn xuất hiện vì nó do HTML tạo ra, nhưng hành động đổi tiêu đề không còn hoạt động. Sau đó đổi lại tên ban đầu.

Bước 10. Mở DevTools để quan sát

Nhấn F12, chọn tab **Elements** để xem HTML, chọn tab **Styles** để xem CSS đang áp dụng, và có thể xem tab **Console** để kiểm tra hoạt động của JavaScript nếu cần.

2.2.3. Liên hệ với phát triển web phía server

Sau khi tìm hiểu vai trò của client, trình duyệt và ba công nghệ nền tảng ở phía người dùng là HTML, CSS và JavaScript, chúng ta cần tiếp tục mở rộng góc nhìn sang **phía server** để hiểu đầy đủ bản chất của một hệ thống web. Nếu chỉ quan sát ở phía client, chúng ta mới thấy được phần giao diện hiển thị và các tương tác trực tiếp trên màn hình. Tuy nhiên, để một website vận hành như một hệ thống hoàn chỉnh, đặc biệt là các website có đăng nhập, tìm kiếm, quản lý dữ liệu, đặt hàng, gửi biểu mẫu hoặc cập nhật nội dung động, thì phía server giữ vai trò trung tâm trong việc xử lý nghiệp vụ và cung cấp dữ liệu.

Trong phát triển web, **phía server** là môi trường xử lý nằm ở máy chủ. Đây là nơi tiếp nhận yêu cầu do client gửi đến, thực hiện các thao tác cần thiết và trả về kết quả tương ứng. Nếu phía client là nơi trình bày nội dung và tiếp nhận hành động của người dùng, thì phía server là nơi chịu trách nhiệm về logic xử lý, lưu trữ dữ liệu, kiểm soát truy cập và điều phối hoạt động của toàn hệ thống. Mỗi liên hệ đầu tiên giữa client và server thể hiện ở **dòng chảy yêu cầu và phản hồi**. Trình duyệt không tự có dữ liệu về sản phẩm, tài khoản, bài viết, lịch học hay kết quả tìm kiếm. Những dữ liệu này thường được quản lý ở phía server. Khi người dùng thực hiện một thao tác trên giao diện, chẳng hạn mở một trang chi tiết sản phẩm hoặc gửi biểu mẫu đăng ký, trình duyệt sẽ gửi yêu cầu đến máy chủ. Server tiếp nhận yêu cầu, phân tích thông tin đi kèm, thực hiện xử lý cần thiết rồi trả về phản hồi. Phản hồi đó có thể là một trang HTML hoàn chỉnh, một phần nội dung cập nhật, hoặc một gói dữ liệu để client tiếp tục hiển thị. Như vậy, phía client là nơi khởi phát thao tác, còn phía server là nơi tạo ra kết quả có giá trị xử lý.

Liên hệ thứ hai nằm ở **vai trò của dữ liệu**. Trong các website tĩnh đơn giản, nội dung có thể được lưu sẵn trong các tệp HTML và gửi trực tiếp cho trình duyệt. Tuy nhiên, phần lớn các hệ thống web hiện đại không hoạt động theo cách cố định như vậy. Nội dung thường được tạo ra dựa trên dữ liệu đang lưu trong cơ sở dữ liệu. Ví dụ, khi người dùng truy cập một trang tin tức, danh sách bài viết không nhất thiết đã được viết cứng trong HTML từ trước, mà có thể được server truy xuất từ cơ sở dữ liệu rồi tạo thành nội dung phù hợp tại thời điểm yêu cầu. Điều

này cho thấy phía server không chỉ là nơi “gửi tệp”, mà còn là nơi **tạo sinh nội dung động** dựa trên dữ liệu và quy tắc nghiệp vụ.

Một liên hệ quan trọng khác là **xử lý logic nghiệp vụ**. Giao diện ở phía client có thể cho phép người dùng nhập thông tin, chọn chức năng và nhấn nút thao tác, nhưng những quyết định nghiệp vụ quan trọng không nên đặt hoàn toàn ở phía trình duyệt. Chẳng hạn, việc kiểm tra tài khoản đăng nhập có hợp lệ hay không, người dùng có quyền truy cập dữ liệu nào, đơn hàng có đủ điều kiện thanh toán hay không, hoặc bản ghi có được phép cập nhật hay không, đều cần được kiểm soát ở phía server. Nguyên nhân là phía server có khả năng quản lý tập trung, bảo đảm tính nhất quán và hạn chế rủi ro do can thiệp không hợp lệ từ phía client.

Mối liên hệ tiếp theo là **bảo mật và an toàn dữ liệu**. Trình duyệt là môi trường mà người dùng có thể nhìn thấy, tương tác và trong nhiều trường hợp có thể kiểm tra, sửa đổi hoặc giả lập yêu cầu. Vì vậy, không thể xem client là môi trường tuyệt đối tin cậy. Những thông tin quan trọng như xác thực đăng nhập, phân quyền người dùng, truy cập cơ sở dữ liệu, xử lý thanh toán hoặc lưu trữ hồ sơ cần được đặt ở phía server. Điều này giúp hệ thống giảm nguy cơ lộ logic nhạy cảm, đồng thời bảo vệ dữ liệu trung tâm tốt hơn. Do đó, khi liên hệ giữa phát triển web phía client và phía server, chúng ta cần hiểu rằng client tạo sự thuận tiện trong sử dụng, nhưng server mới là nền tảng bảo vệ và duy trì trật tự vận hành của hệ thống.

Ngoài xử lý nghiệp vụ và bảo mật, phía server còn liên hệ trực tiếp với phía client thông qua **các cách tổ chức phản hồi**. Trong một số hệ thống, server trả về toàn bộ trang HTML để trình duyệt hiển thị ngay. Đây là cách làm phổ biến trong nhiều ứng dụng web truyền thống. Trong các hệ thống hiện đại hơn, server có thể chỉ trả về dữ liệu ở dạng JSON hoặc XML, còn phía client dùng JavaScript để dựng giao diện. Dù cách tổ chức khác nhau, bản chất vẫn không thay đổi: server cung cấp thông tin hoặc kết quả xử lý, còn client tiếp nhận và thể hiện kết quả đó trước người dùng. Việc hiểu được hai mô hình này giúp chúng ta hình thành tư duy rõ ràng hơn khi học các công nghệ như ASP.NET Core, Node.js, PHP, Java Spring hoặc các hệ thống web dùng API.

Phía server cũng giữ vai trò rất lớn trong **quản lý trạng thái làm việc** của người dùng. Web về bản chất hoạt động theo các yêu cầu rời rạc, nhưng trong thực tế, hệ thống vẫn cần biết ai đang đăng nhập, ai có giỏ hàng nào, ai đang thao tác trên dữ liệu nào và phiên làm việc hiện tại có còn hiệu lực hay không. Những vấn đề đó thường được xử lý ở phía server thông qua session, token, cookie, cơ chế xác thực hoặc các chiến lược quản lý phiên làm việc khác. Nếu chỉ có phía

client mà không có cơ chế server phù hợp, website khó có thể duy trì được các chức năng cá nhân hóa hoặc các quy trình làm việc liên tục.

Như vậy, liên hệ với phát triển web phía server có thể được hiểu theo một tinh thần cốt lõi: phía client chịu trách nhiệm về giao diện và tương tác trực tiếp, còn phía server chịu trách nhiệm về dữ liệu, nghiệp vụ, bảo mật và điều phối hệ thống. Hai phía không thay thế lẫn nhau mà phối hợp để tạo nên một ứng dụng web hoàn chỉnh, ổn định và có khả năng phục vụ người dùng trong những tình huống thực tế đa dạng. Hiểu rõ mối liên hệ này sẽ giúp chúng ta xây dựng nền tảng tư duy đúng đắn trước khi bước sang các nội dung học sâu hơn về kiến trúc ứng dụng web và lập trình phía máy chủ.

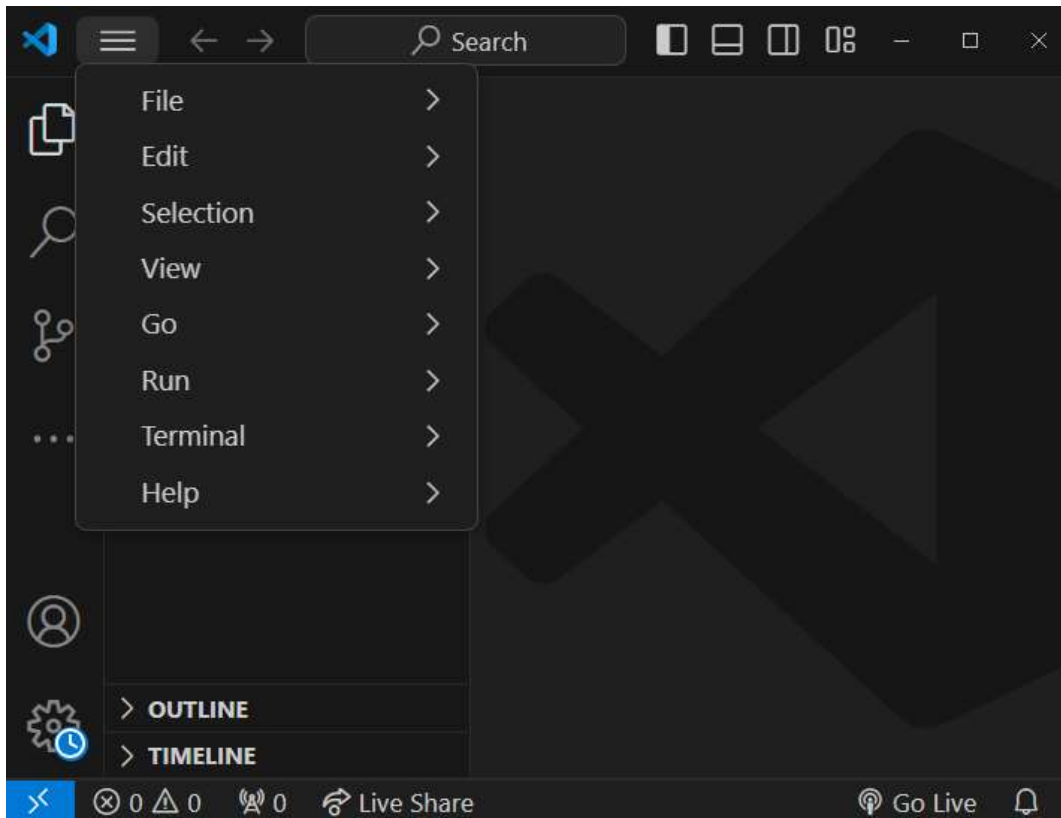
2.3. Công cụ và môi trường thiết kế web

Để học và thực hành thiết kế web hiệu quả, chúng ta cần chuẩn bị đúng **công cụ làm việc** và **môi trường phát triển cơ bản**. Ở giai đoạn nhập môn, mục tiêu không phải là dùng quá nhiều phần mềm phức tạp, mà là chọn những công cụ đủ mạnh, dễ dùng, dễ quan sát lỗi và thuận tiện cho việc thực hành từng bước. Một môi trường học tập hợp lý sẽ giúp chúng ta viết mã nhanh hơn, kiểm tra kết quả rõ hơn và hình thành tác phong làm việc có tổ chức ngay từ đầu.

2.3.1. Lựa chọn trình soạn thảo mã nguồn (VS Code hoặc tương đương)

Trình soạn thảo mã nguồn là công cụ dùng để tạo và chỉnh sửa các tệp HTML, CSS, JavaScript. Đây là nơi chúng ta thực hiện phần lớn công việc khi học thiết kế web. Một trình soạn thảo phù hợp cần có khả năng tô màu cú pháp, gợi ý mã, hỗ trợ quản lý tệp và giúp phát hiện lỗi định dạng cơ bản.

Trong thực tế, **Visual Studio Code (VS Code)** là lựa chọn rất phổ biến vì nhẹ, miễn phí, dễ cài đặt và hỗ trợ tốt cho phát triển web. Ngoài VS Code, chúng ta cũng có thể dùng các công cụ tương đương như Sublime Text, Notepad++, WebStorm hoặc các trình soạn thảo trực tuyến. Tuy nhiên, ở mức học tập cơ bản, VS Code thường là lựa chọn cân bằng tốt giữa tính năng và độ dễ tiếp cận.



Hình 1.3: Môi trường Visual Studio Code

Khi lựa chọn trình soạn thảo, cần ưu tiên các tiêu chí sau. Thứ nhất, giao diện rõ ràng, dễ thao tác với thư mục và tệp. Thứ hai, hỗ trợ tốt cho HTML, CSS, JavaScript. Thứ ba, có khả năng mở rộng bằng tiện ích nếu cần. Thứ tư, chạy ổn định trên máy tính cá nhân của chúng ta.

Các bước chuẩn bị với VS Code

Bước 1. Tải và cài đặt VS Code trên máy tính.

Bước 2. Mở phần mềm và làm quen với các khu vực chính như thanh công cụ, khu vực Explorer, vùng soạn thảo và Terminal.

Bước 3. Tạo một thư mục dự án mới và mở thư mục đó trong VS Code.

Bước 4. Tạo các tệp cơ bản như index.html, style.css, script.js.

Bước 5. Gõ thử một đoạn mã HTML đơn giản để kiểm tra môi trường làm việc.

2.3.2. Sử dụng trình duyệt hiện đại và công cụ dành cho nhà phát triển (Developer Tools)

Sau khi viết mã nguồn, chúng ta cần một công cụ để **chạy**, **quan sát** và **kiểm tra** kết quả. Công cụ đó chính là trình duyệt web hiện đại như Google Chrome, Microsoft Edge hoặc Mozilla Firefox. Trình duyệt không chỉ hiển thị trang web, mà còn là môi trường thực thi HTML, CSS, JavaScript ở phía client.



Hình 1.4: Developer Tools trong Chrome

Bên trong các trình duyệt hiện đại còn có **Developer Tools**. Đây là bộ công cụ rất quan trọng giúp chúng ta kiểm tra cấu trúc HTML, xem CSS đang áp dụng, thử chỉnh sửa giao diện, theo dõi lỗi JavaScript và quan sát các yêu cầu mạng. Trong giảng dạy thiết kế web, Developer Tools là công cụ gần như bắt buộc, vì nó giúp người học nhìn thấy trực tiếp điều gì đang xảy ra bên trong trang web.

Một số khu vực quan trọng trong Developer Tools

- **Elements** dùng để xem cấu trúc HTML và CSS đang áp dụng.
- **Console** dùng để xem lỗi JavaScript và thử lệnh nhanh.
- **Network** dùng để quan sát các tệp mà trình duyệt đã tải như HTML, CSS, JS, hình ảnh.
- **Sources** hỗ trợ xem mã nguồn và gỡ lỗi JavaScript.
- **Application** hỗ trợ quan sát cookie, local storage và một số dữ liệu phía client.

Các bước sử dụng cơ bản

Bước 1. Mở tệp index.html bằng trình duyệt.

Bước 2. Nhấn phím F12 hoặc chuột phải chọn **Inspect** để mở Developer Tools.

Bước 3. Chọn tab **Elements** để kiểm tra các phần tử HTML.

Bước 4. Chọn một phần tử bất kỳ và quan sát CSS áp dụng ở khung bên phải.

Bước 5. Thử thay đổi một giá trị CSS như màu chữ hoặc khoảng cách để xem hiệu ứng trực tiếp.

Bước 6. Mở tab **Console** để kiểm tra xem có lỗi JavaScript hay không.

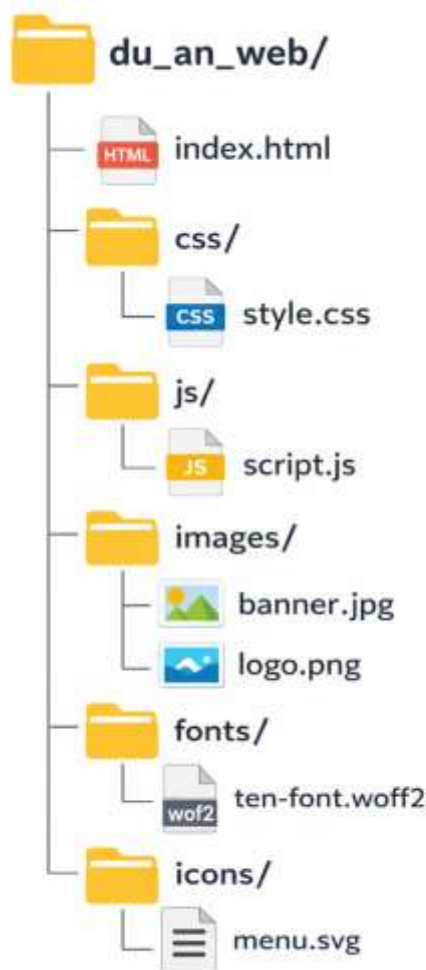
Bước 7. Mở tab **Network**, tải lại trang và quan sát các tệp được trình duyệt tải.

2.3.3. Tổ chức thư mục dự án, tệp HTML, CSS, JS và tài nguyên (hình ảnh, font, icon)

Một dự án web dù đơn giản hay phức tạp đều cần được tổ chức thư mục hợp lý. Việc sắp xếp khoa học giúp chúng ta dễ tìm tệp, dễ sửa lỗi, dễ mở rộng dự án và tránh nhầm lẫn trong quá trình học. Ngay từ đầu, người học nên hình thành thói quen tổ chức rõ ràng thay vì để toàn bộ tệp lẫn lộn trong một thư mục duy nhất.

Ở mức cơ bản, một dự án web thường có một tệp HTML chính, một hoặc nhiều tệp CSS, một hoặc nhiều tệp JavaScript và thư mục chứa tài nguyên như hình ảnh, font chữ, biểu tượng.

Ví dụ cấu trúc thư mục



Hình 1.5: Cấu trúc thư mục

Trong cấu trúc này, index.html là điểm bắt đầu của trang web. Thư mục css chứa các tệp định kiểu. Thư mục js chứa mã JavaScript. Thư mục images chứa

hình ảnh. Thư mục fonts chứa phông chữ tùy chỉnh. Thư mục icons chứa biểu tượng.

Nguyên tắc tổ chức cơ bản cấu trúc thư mục như sau:

- Tên thư mục và tệp nên ngắn gọn, dễ hiểu, không đặt tùy tiện.
- Nên tách riêng từng loại tài nguyên theo đúng chức năng.
- Các đường dẫn trong HTML cần viết chính xác theo vị trí thư mục.
- Khi dự án lớn hơn, có thể chia nhỏ tiếp theo từng khu vực chức năng.

Các bước thực hiện

Bước 1. Tạo một thư mục gốc cho dự án, ví dụ du_an_web.

Bước 2. Trong thư mục gốc, tạo các thư mục con như css, js, images, fonts, icons.

Bước 3. Tạo tệp index.html ở thư mục gốc.

Bước 4. Tạo tệp style.css trong thư mục css.

Bước 5. Tạo tệp script.js trong thư mục js.

Bước 6. Đặt các hình ảnh minh họa vào thư mục images.

Bước 7. Liên kết đúng đường dẫn trong index.html.

Ví dụ liên kết tài nguyên

```
<link rel="stylesheet" href="css/style.css">
<script src="js/script.js"></script>

```

2.4. Trang web đầu tiên

Trong bài học đầu tiên về thực hành thiết kế web, mục tiêu quan trọng nhất là giúp chúng ta tự tạo được một trang web tối giản, mở được trên trình duyệt và nhận biết rõ mối liên hệ giữa tệp HTML, CSS, JavaScript với kết quả hiển thị. Ở giai đoạn này, chúng ta chưa đi sâu vào giao diện phức tạp hay xử lý tương tác nâng cao, mà tập trung xây dựng đúng quy trình kỹ thuật cơ bản: tạo tệp, viết mã, mở trên trình duyệt và kiểm tra bằng công cụ dành cho nhà phát triển.

2.4.1. Tạo tệp HTML đơn giản và hiển thị văn bản trên trình duyệt

Tệp HTML là điểm khởi đầu của một trang web. Đây là nơi chứa cấu trúc nội dung mà trình duyệt sẽ đọc và hiển thị. Trong bài thực hành đầu tiên, chúng ta cần tạo một tệp HTML tối giản nhưng đúng chuẩn để kiểm tra rằng môi trường làm việc đã sẵn sàng.

Bài thực hành tạo tệp HTML và hiển thị

Bước 1. Tạo một thư mục mới, ví dụ trang_web_dau_tien.

Bước 2. Mở thư mục này bằng VS Code hoặc trình soạn thảo mã nguồn tương đương.

Bước 3. Tạo tệp index.html.

Bước 4. Nhập nội dung HTML cơ bản sau:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Trang web đầu tiên</title>
</head>
<body>
  <h1>Xin chào, đây là trang web đầu tiên của tôi</h1>
  <p>Tôi đang học thiết kế web cơ bản.</p>
</body>
</html>
```

Bước 5. Lưu tệp.

Bước 6. Nhấp đúp vào index.html hoặc mở bằng trình duyệt.

Kết quả mong đợi

Trình duyệt hiển thị một tiêu đề lớn và một đoạn văn bản ngắn. Kết quả này xác nhận rằng tệp HTML đã được viết đúng cú pháp cơ bản và trình duyệt đã đọc được nội dung.

2.4.2. Liên kết tệp CSS, JS rộng để chuẩn bị cho các bài học tiếp theo

Sau khi trang HTML hiển thị được nội dung, bước tiếp theo là tổ chức cấu trúc dự án theo hướng đúng kỹ thuật. Dù ở bài đầu tiên chúng ta chưa viết CSS và JavaScript thực sự, việc tạo sẵn các tệp này và liên kết với HTML sẽ giúp chúng ta hiểu sớm cách các thành phần phía client phối hợp với nhau.

Thực hành liên kết tệp CSS, JS

Bước 1. Trong thư mục dự án, tạo hai thư mục con: css và js.

Bước 2. Trong thư mục css, tạo tệp style.css.

Bước 3. Trong thư mục js, tạo tệp script.js.

Bước 4. Để hai tệp này rộng hoặc có thể thêm chú thích đơn giản như sau:

Tệp style.css:

```
/* Tệp CSS chuẩn bị cho các bài học tiếp theo */
```

Tệp script.js:

// Tập JavaScript chuẩn bị cho các bài học tiếp theo

Bước 5. Quay lại tệp index.html và bổ sung liên kết trong phần <head> và cuối phần <body>:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Trang web đầu tiên</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Xin chào, đây là trang web đầu tiên của tôi</h1>
  <p>Tôi đang học thiết kế web cơ bản.</p>
  <script src="js/script.js"></script>
</body>
</html>
```

Bước 6. Lưu tất cả các tệp.

Kết quả mong đợi

Trang web vẫn hiển thị bình thường như trước. Dù CSS và JavaScript chưa có nội dung xử lý thực tế, trình duyệt đã nhận biết được rằng trang HTML này có liên kết đến hai tệp ngoài. Đây là bước chuẩn bị đúng kỹ thuật cho các bài học tiếp theo về định dạng giao diện và lập trình tương tác.

2.4.3. Kiểm tra hiển thị trên trình duyệt, sử dụng Developer Tools để xem cấu trúc trang

Sau khi tạo trang web đầu tiên, chúng ta cần biết cách kiểm tra kết quả không chỉ bằng mắt nhìn, mà còn bằng công cụ kỹ thuật của trình duyệt. Đây là thói quen rất quan trọng trong học tập và phát triển web, vì nó giúp quan sát cấu trúc HTML thực tế, kiểm tra liên kết tài nguyên và phát hiện lỗi nhanh hơn.

Thực hành kiểm tra trên trình duyệt và sử dụng Developer Tools

Bước 1. Mở index.html trên trình duyệt.

Bước 2. Kiểm tra bằng mắt thường xem tiêu đề và đoạn văn đã hiển thị đúng chưa.

Bước 3. Nhấn phím F12 hoặc nhấp chuột phải trên trang, chọn **Inspect** để mở Developer Tools.

Bước 4. Chọn tab **Elements**.

Bước 5. Quan sát cấu trúc HTML mà trình duyệt đang hiển thị, gồm các phần như `<html>`, `<head>`, `<body>`, `<h1>`, `<p>`.

Bước 6. Nhấp vào phần tử `<h1>` trong tab Elements và đối chiếu với nội dung đang hiện trên trang.

Bước 7. Mở tab **Network**, sau đó tải lại trang bằng phím F5.

Bước 8. Quan sát rằng trình duyệt đã tải các tệp `index.html`, `style.css` và `script.js`.

Bước 9. Mở tab **Console** để kiểm tra xem có lỗi nào xuất hiện hay không.

Kết quả mong đợi

Chúng ta nhìn thấy rõ ba điều. Thứ nhất, trình duyệt đã dựng đúng cấu trúc HTML. Thứ hai, các tệp CSS và JS đã được liên kết thành công. Thứ ba, Developer Tools là công cụ quan trọng để quan sát và kiểm tra hoạt động của trang web.

Tổng kết

Bài mở đầu đã giúp người học hình thành bức tranh nền tảng của thiết kế web. Trước hết, chúng ta phân biệt được Internet, World Wide Web, website và trang web; đồng thời nhận biết được các thành phần quen thuộc như tên miền, URL, hosting, máy chủ web và trình duyệt.

Trên cơ sở đó, bài học làm rõ mô hình client - server: trình duyệt gửi yêu cầu, máy chủ tiếp nhận và phản hồi nội dung để trang web được hiển thị cho người dùng. Trong phía client, HTML đảm nhiệm vai trò tổ chức cấu trúc nội dung, CSS phụ trách trình bày và bố cục, còn JavaScript tạo nên sự tương tác và hành vi động của giao diện.

Về mặt thực hành, người học đã biết lựa chọn và thiết lập môi trường làm việc gồm trình soạn thảo mã nguồn, trình duyệt hiện đại và Developer Tools; biết tổ chức thư mục dự án ban đầu; tạo được tệp HTML đầu tiên; liên kết trước các tệp CSS và JavaScript; đồng thời quan sát cấu trúc trang ngay trên trình duyệt để chuẩn bị cho việc học sâu hơn ở các bài sau.

Sau khi hiểu được web vận hành như thế nào và đã tạo được trang đầu tiên, người học bước sang Bài 1 để đi sâu vào ngôn ngữ HTML - nền tảng cốt lõi của thiết kế web. Nếu Bài mở đầu trả lời câu hỏi “trang web hoạt động trong môi trường nào”, thì Bài 1 sẽ trả lời trực tiếp câu hỏi “nội dung trang web được cấu trúc ra sao”.

Bài tập và đánh giá

A. Câu hỏi lý thuyết (5 điểm)

Câu 1 (1 điểm)

Trình bày khái niệm Internet, World Wide Web, website và trang web. Phân biệt ngắn gọn các khái niệm này bằng một ví dụ thực tế.

Câu 2 (1 điểm)

Tên miền, hosting, máy chủ web và địa chỉ URL có vai trò gì trong việc đưa một website lên môi trường truy cập công khai?

Câu 3 (1 điểm)

Mô tả mối quan hệ giữa client và server trong thiết kế web. Trình bày luồng cơ bản khi người dùng nhập một địa chỉ web trên trình duyệt.

Câu 4 (1 điểm)

Phân tích vai trò của HTML, CSS và JavaScript trong phía client. Vì sao ba thành phần này thường đi cùng nhau trong một dự án web?

Câu 5 (1 điểm)

Nêu các công cụ và bước chuẩn bị tối thiểu để bắt đầu học thiết kế web. Giải thích ngắn gọn lợi ích của Developer Tools đối với người mới học.

B. Câu hỏi thực hành (5 điểm)

Thực hành 1 (2,5 điểm)

Yêu cầu: Thiết lập môi trường làm việc cho một dự án web cơ bản.

- Tạo một thư mục dự án có cấu trúc tối thiểu gồm: index.html, css/style.css, js/script.js và thư mục images.
- Mở dự án bằng VS Code hoặc trình soạn thảo tương đương; chạy trang bằng trình duyệt hiện đại.
- Mở Developer Tools để kiểm tra tiêu đề trang, cấu trúc phần tử body và các tệp liên kết ban đầu.
- Chụp hoặc ghi lại minh chứng các bước đã thực hiện.

Thực hành 2 (2,5 điểm)

Yêu cầu: Xây dựng trang web đầu tiên và tạo bước đệm sang Bài 1 về HTML.

- Tạo nội dung cho tệp index.html gồm tiêu đề trang, một heading chính, một đoạn giới thiệu ngắn và một danh sách liệt kê ít nhất 3 nội dung sẽ học trong Bài 1.
- Liên kết trước tệp css/style.css và js/script.js vào trang HTML dù hai tệp này có thể đang ở trạng thái khởi tạo đơn giản.

- Bổ sung ít nhất 2 liên kết giả định hoặc liên kết nội bộ để thể hiện định hướng mở rộng sang website nhiều trang ở Bài 1.
- Dùng Developer Tools kiểm tra phần head, body và xác nhận trang hiển thị đúng trên trình duyệt.

C. Bảng đánh giá (thang 10 điểm)

C.1. Phân bố điểm

Nhóm	Nội dung	Điểm
Lý thuyết	Câu 1 - Câu 5	5,0
Thực hành	Thực hành 1	2,5
Thực hành	Thực hành 2	2,5
Tổng		10,0

C.2. Chấm câu lý thuyết (mỗi câu tối đa 1 điểm)

Mức	Điểm	Tiêu chí	Gợi ý lỗi thường gặp
Đạt tối đa	1,0	Trả lời đúng trọng tâm, đủ ý chính, dùng thuật ngữ tương đối chính xác và có ví dụ hoặc diễn giải phù hợp với bài học về web, client - server, công cụ và môi trường thiết kế.	Nhầm lẫn giữa Internet và Web; mô tả chưa rõ vai trò HTML/CSS/JavaScript; giải thích mơ hồ về hosting, domain hoặc Developer Tools.

Đạt một phần	0,5	Nêu được ý chính nhưng còn thiếu chi tiết, ví dụ chưa rõ hoặc chưa liên hệ đúng bối cảnh thiết kế web cơ bản.	Trả lời chung chung; thiếu phân biệt giữa website và trang web; chưa mô tả được luồng client - server hoặc công cụ thực hành.
Chưa đạt	0,0	Sai trọng tâm hoặc bỏ sót phần lớn nội dung yêu cầu.	Nhầm lẫn khái niệm cơ bản, không nêu được vai trò của các thành phần chính.

C.3. Chấm Thực hành 1 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Chưa đạt	Minh chứng cần có
Cấu trúc thư mục và tệp dự án	0,5	Tạo đúng cấu trúc thư mục, đặt tên tệp hợp lý.	Có cấu trúc nhưng thiếu 1 thành phần nhỏ hoặc sắp xếp chưa gọn.	Thiếu nhiều tệp/thư mục cơ bản.	Ảnh chụp cấu trúc thư mục hoặc màn hình VS Code.
Thiết lập công cụ	0,5	Mở và thao tác được với VS Code, trình duyệt, Developer Tools.	Mở được công cụ nhưng thao tác còn thiếu một phần.	Chưa mở hoặc chưa sử dụng được công cụ yêu cầu.	Ảnh chụp cửa sổ làm việc, tab DevTools.

Liên kết và chạy trang	0,75	Trang mở được trên trình duyệt, liên kết CSS/JS đúng vị trí.	Trang mở được nhưng liên kết còn thiếu hoặc sai nhỏ.	Trang không chạy đúng hoặc tệp liên kết sai.	Ảnh chụp trình duyệt và mã nguồn.
Tính gọn gàng, đúng quy trình	0,75	Thao tác đúng trình tự, thư mục gọn, tên tệp rõ ràng.	Có thực hiện nhưng trình bày chưa gọn hoặc thiếu minh chứng.	Thiếu quy trình, làm việc rời rạc.	Bản mô tả ngắn hoặc ảnh minh chứng các bước.

C.4. Chấm Thực hành 2 (tối đa 2,5 điểm)

Tiêu chí	Điểm tối đa	Đạt tối đa	Đạt một phần	Chưa đạt	Minh chứng cần có
Cấu trúc cơ bản của trang HTML	0,75	Có tiêu đề trang, heading, đoạn văn và nội dung hiển thị rõ.	Có trang HTML nhưng thiếu một thành phần nhỏ.	Thiếu cấu trúc cơ bản của trang.	Mã nguồn index.html và ảnh kết quả hiển thị.
Khả năng chuẩn bị cho Bài 1	0,5	Có liên kết CSS/JS và định hướng mở rộng sang nhiều trang.	Có chuẩn bị nhưng chưa đầy đủ hoặc liên kết còn sơ sài.	Không thể hiện được bước chuyển tiếp sang Bài 1.	Mã nguồn phần head và các liên kết.
Tổ chức nội dung	0,5	Nội dung ngắn gọn,	Có nội dung	Nội dung quá thiếu,	Ảnh chụp trang hiển

HTML ban đầu		dễ hiểu, có danh sách học tập cho Bài 1.	nhưng còn sơ lược hoặc thiếu mạch lạc.	chưa thể hiện ý đồ học tập.	thị và phần body HTML.
Kiểm tra bằng Developer Tools	0,75	Biết dùng DevTools để kiểm tra head, body và tài nguyên liên kết.	Có mở DevTools nhưng kiểm tra chưa đầy đủ.	Không có thao tác kiểm tra hoặc không có minh chứng.	Ảnh chụp DevTools hoặc mô tả kết quả kiểm tra.